

Universidad Autonóma del Estado de Hidalgo

Instituto de Ciencias Básicas en Ingenerías

Implementación de Backtracking para la Solución y Unicidad de Sudokus en Python.

Proyecto, tercera evaluación

Presentada por:

Bautista Romero Arizbeth Guadalupe
Sánchez Martínez Jared

Mineral de la Reforma, Hidalgo
21 de Febrero, 2026

Introducción

El Sudoku es un juego de lógica que consiste en completar una cuadrícula de 9×9 con números del 1 al 9, respetando las siguientes reglas:

- Cada fila debe contener los números del 1 al 9 sin repetir.
- Cada columna debe contener los números del 1 al 9 sin repetir.
- Cada subcuadrícula de 3×3 debe contener los números del 1 al 9 sin repetir.

El problema computacional consiste en encontrar una solución válida para un tablero incompleto. Para resolver este problema se utiliza un algoritmo de *backtracking*, una técnica recursiva ampliamente utilizada en problemas de búsqueda y optimización.

Objetivo del Código

El código implementado en Python tiene como propósito:

- Verificar si un tablero de Sudoku es válido.
- Resolver automáticamente un Sudoku.
- Determinar si el tablero posee más de una solución.

Descripción General del Algoritmo

El algoritmo principal utiliza la técnica de *backtracking*. Esta técnica funciona probando posibles valores en las casillas vacías del tablero.

Funcionamiento General

1. Buscar una casilla vacía.
2. Intentar colocar un número del 1 al 9.
3. Verificar si el número cumple las reglas del Sudoku.
4. Si el número es válido, continuar recursivamente.
5. Si en algún momento no existe una opción válida, regresar atrás y probar otro número.

Funciones Principales

Impresión del tablero

La función `print_grid()` imprime el tablero completo en pantalla.

```
def print_grid(arr):
```

donde arr, es la matriz dada de 9×9 inicialmente.

Búsqueda de casillas vacías

La función `find_empty_location()` localiza una casilla cuyo valor sea cero, lo que representa un espacio vacío.

```
def find_empty_location(arr, l):
```

Verificación de restricciones

El programa cuenta con funciones que revisan si un número ya existe en:

- Una fila.
- Una columna.
- Una subcuadrícula de 3×3.

Estas funciones son:

```
used_in_row()
used_in_col()
used_in_box()
```

Validación de posiciones

La función `check_location_is_safe()` verifica si un número puede colocarse en una posición específica sin violar las reglas del Sudoku.

```
def check_location_is_safe(arr, row, col, num):
```

Validación del tablero

La función `is_valid()` revisa si el tablero inicial cumple correctamente las restricciones del juego.

```
def is_valid(tablero):
```

Resolución del Sudoku

La función principal del algoritmo es `solve_sudoku()`, la cual implementa el método de backtracking.

```
def solve_sudoku(arr):
```

Esta función prueba diferentes números hasta encontrar una solución válida.

Detección de múltiples soluciones

La función `there_is_another_solution()` analiza si existe más de una solución posible para el tablero.

```
def there_is_another_solution(original, solucion):
```

Importancia del Backtracking

El método de backtracking es útil porque permite explorar sistemáticamente todas las posibilidades hasta encontrar una solución válida. Aunque puede ser costoso computacionalmente en algunos casos, resulta eficiente para problemas como el Sudoku.

Conclusión

El código desarrollado permite resolver automáticamente tableros de Sudoku utilizando técnicas de búsqueda recursiva y validación lógica. Además de encontrar soluciones, también es capaz de verificar si un tablero tiene múltiples soluciones posibles.

Este tipo de algoritmos representa una aplicación práctica de conceptos fundamentales de programación, recursión y teoría de restricciones.

Bibliografía

[1] Knuth, D. E. (2011). *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms*. Addison-Wesley.

[2] Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[4] Herzberg, A. M., & Murty, M. R. (2007). Sudoku squares and chromatic polynomials. *Notices of the AMS*, 54(6), 708–717.

[5] Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python*. Wiley.